



Faculty of Computers and Artificial Intelligence

Cairo University

Graduation Project

Name	ID
Ahmed Mohamed Abdalnaby	20210035
Ehab Ashraf Abdelmalak	20211021
Martina Maazouz sabry	20210309
Aya Osama Mohamed	20210081
Omnya Ahmed Mohamed	20210074

Supervised by: Dr. Reda Abd-El Wahab

March 2025

Table of Contents

1. Introduction....3
2. Scope....3
3. objectives....3
4. Datasets....4
5. Problems....4
6. Competitor applications....4
7. Using Gemini: Advanced Generative AI Integration....10
8. Comparison: Gemini API vs. Transformers vs. Custom Model....11
9. Clustering....12
10. Backend Part....14
11. Frontend Part....27

1. Introduction

KmtKids is an innovative educational app designed to teach young children from 6-12 years old about history and civilization of ancient Egypt in a fun and interactive way. By using voice-based chatbot, storytelling, and interactive quizzes, KmtKids allows children to explore the wonders of ancient Egyptian civilization through engaging and age-appropriate activities. The app aims to make learning history enjoyable and accessible to early learners by blending technology with education. This project leverages AI technologies, including speech recognition and natural language processing and generative-AI, to create an immersive experience for children.

In this project, we adopted the **Agile methodology** to ensure flexibility, iterative progress, and continuous collaboration. Our approach involved breaking the project into smaller, manageable tasks (sprints), conducting regular stand-up meetings, and iterating based on feedback to deliver incremental value. This allowed us to adapt to changes quickly, prioritize deliverables effectively, and maintain a high level of transparency throughout the project lifecycle.

2. Scope

This project focuses on developing an engaging application which is directed to children between the ages of 5 and 12, offering a mix of education and entertainment. The app's core features include dynamic stories, interactive quizzes with a leaderboard to encourage competition, and a chatbot capable of answering questions about Egyptian history using voice input and AI-generated responses as a main feature. We also explore the challenges of designing AI solutions for children, ensuring accuracy, simplicity, and cultural relevance. Additionally, the project aims to evaluate the educational impact of such tools and their potential for global expansion beyond Egyptian history.

3. objectives

The primary objective of KmtKids is to encourage a love for history and learning among children through interactive digital tools. By integrating voice-enabled AI, storytelling, and quizzes, the app aims to:

- Provide a fun and engaging way for children to learn about Egyptian civilization.
- Improve children’s critical thinking and curiosity through quiz challenges and leaderboards.
- Offer personalized learning experiences using a chatbot that adapts to the child’s level of understanding.

- Create a scalable platform that can later include content from other historical civilizations and cultures.

4. Datasets

For now, the model uses the data that's uploaded to it in history

5. Problems

User's point of view

Considering the features of our application, which are free and with good performance, users would find it a great experience to learn or educate their children

Stakeholders' point of view

Any organization can upload their learning material using the application our model learns from this material, then the users can ask questions about any topic in the domain of the uploaded material to the chatbot ,generates stories, and make quizzes.

6. Competitor applications



Figure 5.1: mussila website logo

1. About mussila website

Mussila is a website and platform focused on music education for children. It provides interactive and engaging tools to help kids learn about music through games, activities, and lessons. The platform aims to make learning music fun and accessible, often incorporating elements like rhythm and melody into its curriculum

Website Link: mussila



Figure 6.1: mussila website insights

2. Learner User flow (Flowchart)

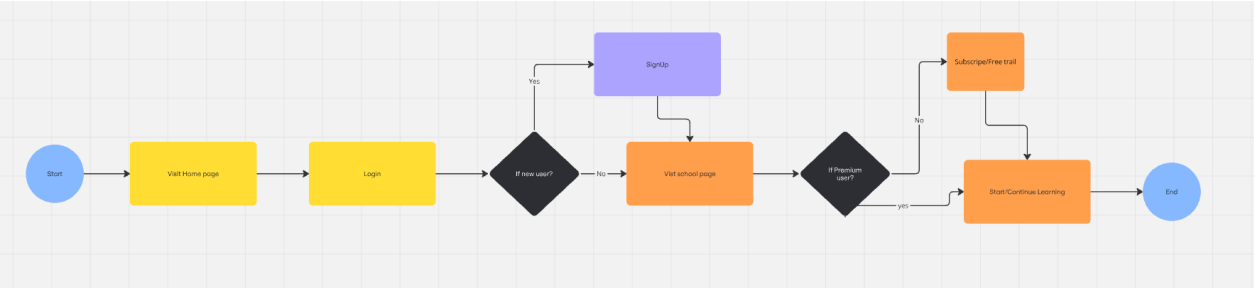


***Figure 6.2: mussila website Lerner* flow**

3. Strength Points

Mussila offers two paths of learning

1- Mussila Word play: Mussila Word Play is an interactive game that enhances children's vocabulary through music. By integrating rhythm and melody, it makes learning enjoyable and memorable. The game features fun challenges, bright visuals, and age-appropriate content, promoting language skills and creativity

2- mussila music: Mussila Music is an engaging platform designed to teach children the fundamentals of music through interactive activities and games. It covers essential concepts like rhythm, melody, and music theory in a fun and accessible way. With a user-friendly interface, it encourages young learners to explore and develop their musical abilities at their own pace

4. weakness Points

1- No AI used

2- No free plans with Limits, only free trial for 7 days

3- No age match games and lessons

5. Our Competitive Advantage

1- We are using AI to mentor children and answer them questions with generative AI

2- Our application is totally free

3- Get age match answers



Figure 7.1: curious world website logo

1. About curious world website

This website offers interactive learning on various topics, including history and geography. Children can explore activities, stories, and games with voice instructions and feedback

Website Link: curious world

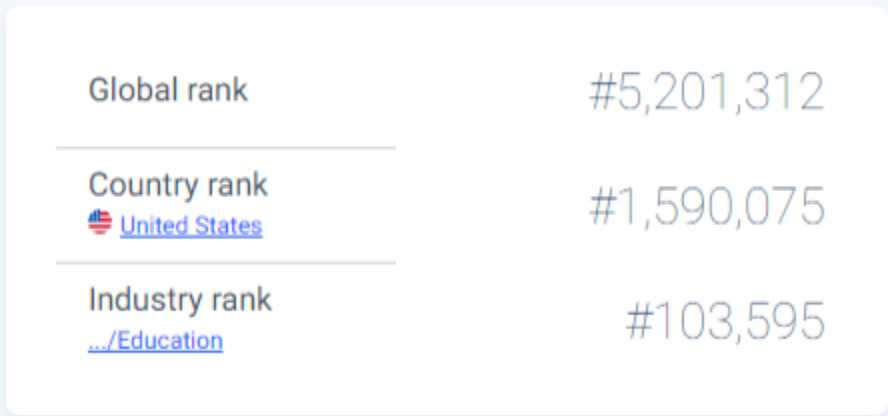


Figure 7.2: curious world website insights

2. Learner User flow (Flowchart)

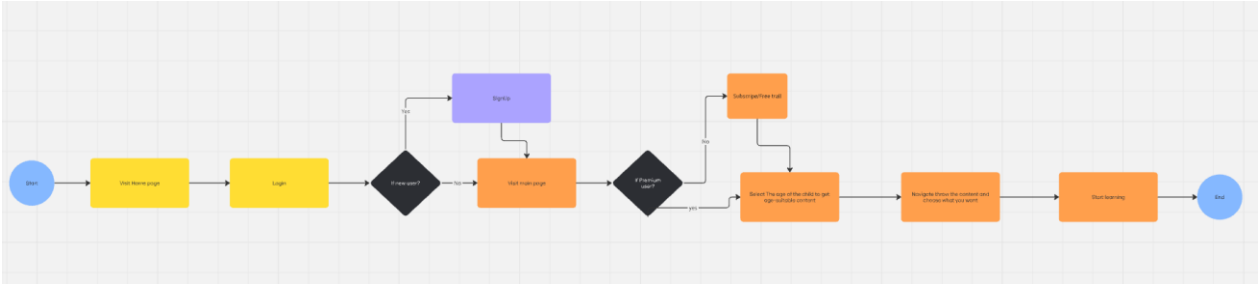


Figure 8.1: curious world website user flow

3. Strength Points

1- Get age match games and lessons

2- Varity in learning ways(play a game, read or watch a video)

3- voice instructions and feedback: when making progress or doing exercise you got congratulations with voice messages

4. Weakness Points

1- No AI used

2- No free plans with Limits, only free trial for 7 days and one demo game

5. Our Competitive Advantage

1- We are using AI to mentor children and answer them questions with generative AI

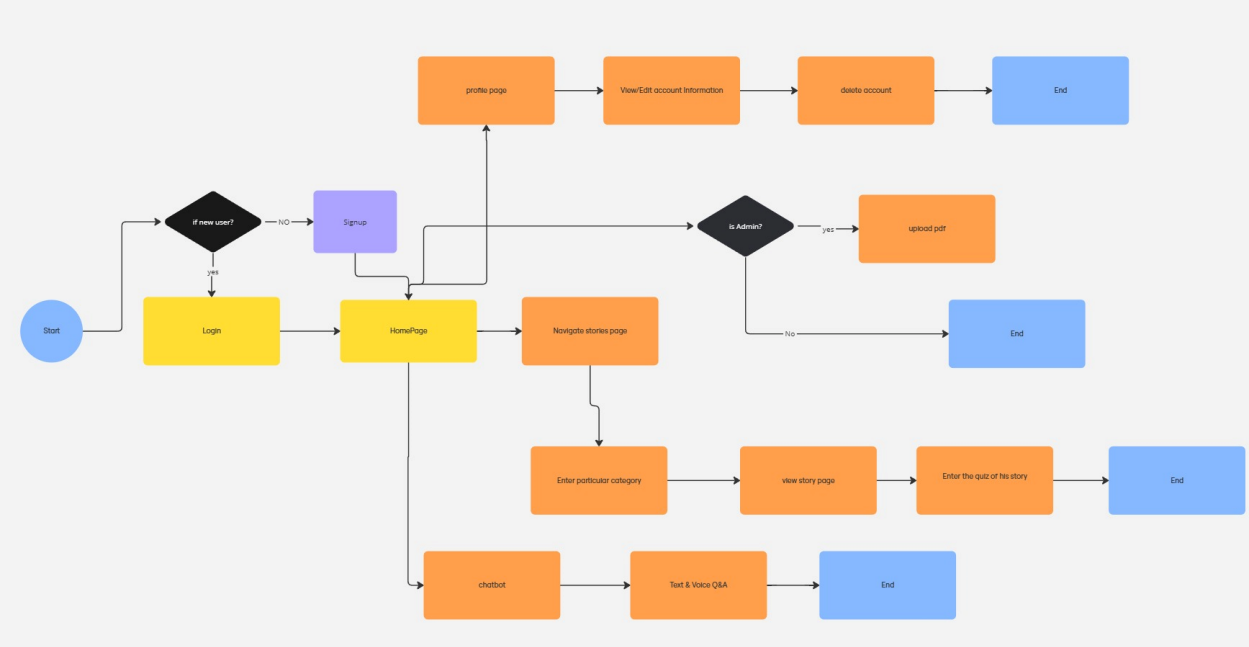
2- Our application is totally free

Our application (KmtKids)

1. About KmtKids

It's a educational application for kids aging 6 to 12 years old, mainly made for children to understand and learn more about the Egyptian civilization, including features like generating stories, and quizzes

2. **Learner User flow (Flowchart)**



3. Strength Points

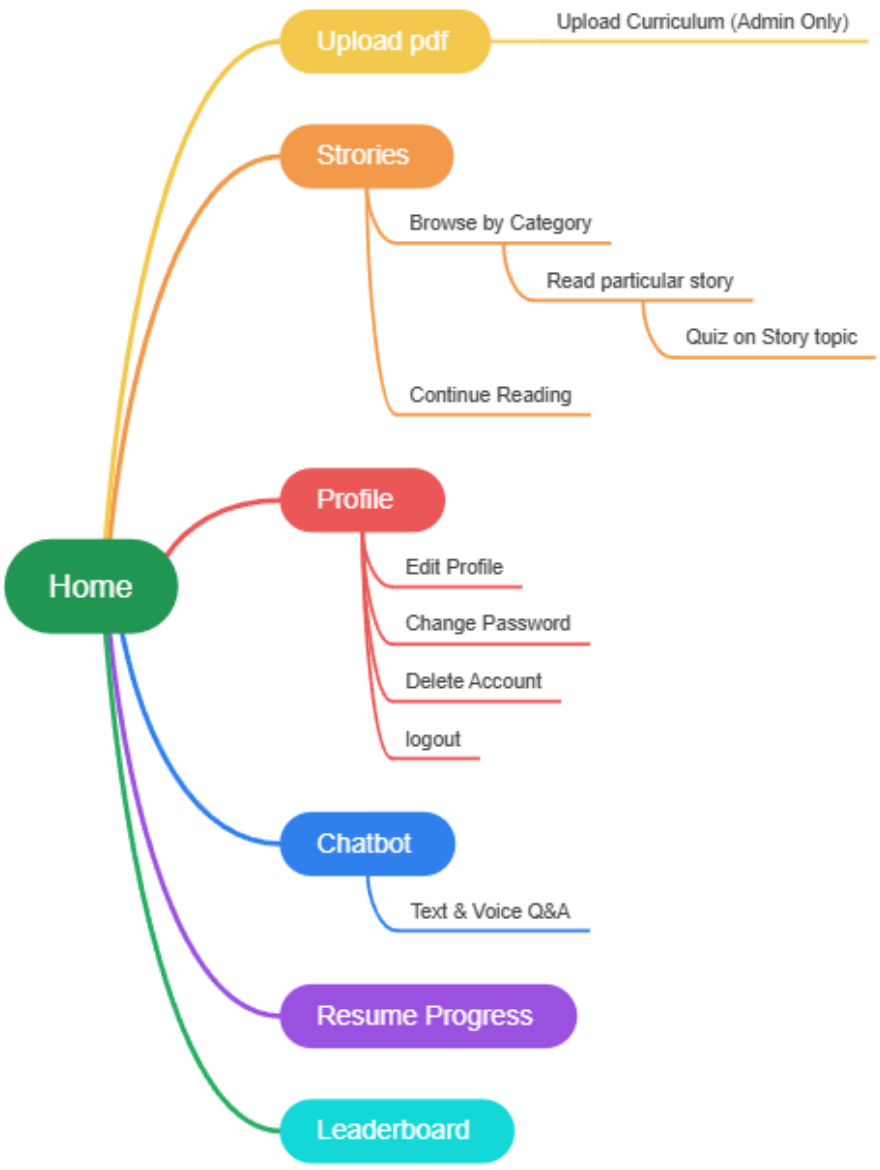
- Unlike any other competitor, a user can enter their age, and the answers will come out according to that age so that the kids can have better understanding
- A feature like uploading learning material can enable any institution to upload the material, our model learns from it then the user can ask the chatbot anything about the domain of the material, generate stories, and make quizzes from the material.

- Completely free
- We use AI to answer questions (chatbot)

4. Weakness Points

- The project currently does not utilize cloud services for hosting, storage, or scalability.

5. Site map



7. Using Gemini: Advanced Generative AI Integration

Functional Components

1. **PDF Text Extraction**
   - Fetches the PDF from a specified URL using HTTP requests.
   - Reads the content using a PDF reading library.
   - Extracts and concatenates text from all pages into a unified string.

2. **Speech-to-Text Conversion**

   - Captures user audio input using a speech recognition library.
   - Recognizes and transcribes Arabic speech.
   - Stores the transcribed text in a text file for later processing.

3. **Natural Language Processing (NLP)**

uses the Google Generative AI API to process and respond to user queries.

   - Takes user queries and appends contextual restrictions to guide the AI's response.
   - Utilizes a chat session feature to maintain historical context based on user input and extracted text.

4. **Text-to-Speech Synthesis**

   The system can convert text queries or responses into an audio format using a text-to-speech library.

   - Converts text into Arabic speech and saves it as an MP3 file.
   - Optionally plays the synthesized speech for auditory feedback.

5. **AI Chat Session Initialization**

   A chat session is created to tailor AI responses to the context of extracted PDF content.

   - Feeds the extracted text from PDFs as context into the AI model.

- Prepares the AI to respond to user queries effectively, leveraging the content of the document.

8. Comparison: Gemini API vs. Transformers vs. Custom Model

Feature	Gemini API	Transformers	Custom Model
Ease of Use	Very easy; cloud-based API with no setup.	Requires installation and model selection; may need fine-tuning.	Most difficult; requires model design, training, and optimization.
Customization & Flexibility	Limited customization.	Highly flexible: choose models, fine-tune, and modify parameters.	Full flexibility over architecture, training, and data.
Performance & Accuracy	High accuracy but not fine-tunable.	Depends on model choice and fine-tuning; specialized models can outperform Gemini.	Performance varies; can exceed both with proper optimization.
Inference Speed & Scalability	Ultra-fast; cloud-based and scales easily.	Small models are fast; large models need GPUs for real-time use.	Speed depends on hardware; scaling is challenging.
Cost & Compute Requirements	Pay-per-use; no hardware needed.	Free for small use; fine-tuning can be costly.	Most expensive; requires GPUs and long training times.
Data Privacy & Security	Data processed on Google's servers.	Can run offline or on private cloud.	Fully private; user owns data and model.

Limitations	The API cannot handle large volumes of files due to usage restrictions. It has a daily usage limit of 1,500 requests. The API does not mark files as "learned," resulting in redundant processing of the same files.	Requires large datasets; high computational needs.	Time-consuming, costly hardware, and complex development.

9. Clustering

Why use clustering in our application

- Mainly for generating stories, the name of the files will be sent from the backend to the model, every couple of files will be used to generate a story from them

Word vector

- Word vectors are numerical representations of words in a continuous vector space. They capture the semantic meaning of words, allowing words with similar meanings to have similar vector representations. Word vectors are typically learned from large text corpora using models like Word2Vec, GloVe, or FastText.
- Dimensionality: Each word is represented as a vector of fixed size (e.g., 300 dimensions in the FastText model).
- Semantic Similarity: Words with similar meanings (e.g., "king" and "queen") will have vectors that are close to each other in the vector space.
- Arithmetic Operations: Word vectors can be used to perform arithmetic operations like addition and subtraction to capture relationships between words

How we used word vector

- We used word vectors from a pre-trained FastText model to represent Arabic words.
- These word vectors are averaged to create a single vector representation for each sentence, which is then used for clustering.

FastText Model to Get Word Vectors

- FastText is a library developed by Facebook AI Research for efficient learning of word representations and sentence classification. Unlike traditional word embedding models like Word2Vec, FastText represents words as a sum of their Subword (n-gram) vectors. This makes it particularly effective for handling morphologically rich languages (like Arabic) and rare words.

Features of FastText

- Subword Information: FastText breaks words into smaller n-grams (e.g., tri-grams) and learns vectors for these subwords. This allows it to generate better representations for rare or misspelled words.
- Pre-trained Models: FastText provides pre-trained word vectors for many languages, including Arabic, which can be downloaded and used directly.
- Efficiency: FastText is computationally efficient and can handle large datasets.

How we used FastText model

- We load a pre-trained FastText model (cc.ar.300.bin) for Arabic.
- The model is used to retrieve word vectors for each word in the sentences.
- These word vectors are averaged to create a single vector representation for each sentence

Mean Shift Algorithm for Clustering

- Mean Shift is a non-parametric clustering algorithm that does not require the number of clusters to be specified in advance. It works by iteratively shifting data points toward the mode (peak) of the data distribution, effectively finding dense regions in the data.

How Mean Shift Works

- Kernel Density Estimation: Mean Shift uses a kernel (e.g., Gaussian) to estimate the density of data points in the feature space.
- Shift Toward Modes: For each data point, the algorithm calculates the mean of the points within a certain radius (bandwidth) and shifts the point toward that mean.
- Convergence: The process repeats until the points converge to the modes of the density function, which become the cluster centers.
- Automatic Cluster Detection: The number of clusters is determined automatically based on the data distribution.

Why choose Mean Shift

- No Need for Predefined Clusters: Unlike K-Means, Mean Shift does not require the number of clusters to be specified.
- Handles Arbitrary Cluster Shapes: Mean Shift can find clusters of any shape, as long as they correspond to dense regions in the data.

- Bandwidth Parameter: The bandwidth parameter controls the size of the region considered for density estimation. A larger bandwidth results in fewer clusters.

How we used Mean Shift Algorithm

- We used Mean Shift to cluster the sentence vectors generated from the FastText model.
- The algorithm groups sentences with similar semantic meanings into the same cluster.
- The output shows which sentences belong to the same cluster, indicating their semantic similarity.

10. Backend Part

1. Introduction

The backend of this educational platform serves as the foundation for managing user interactions, content delivery, and data processing. It is responsible for handling authentication, storing quiz and story progress, and ensuring a smooth learning experience for users. The application leverages MongoDB, Node.js, and Express.js to provide a scalable and efficient backend architecture. Additionally, Fast API is utilized to create APIs for machine learning models, enabling seamless integration of advanced analytics and personalized learning experiences.

2. Core Backend Responsibilities

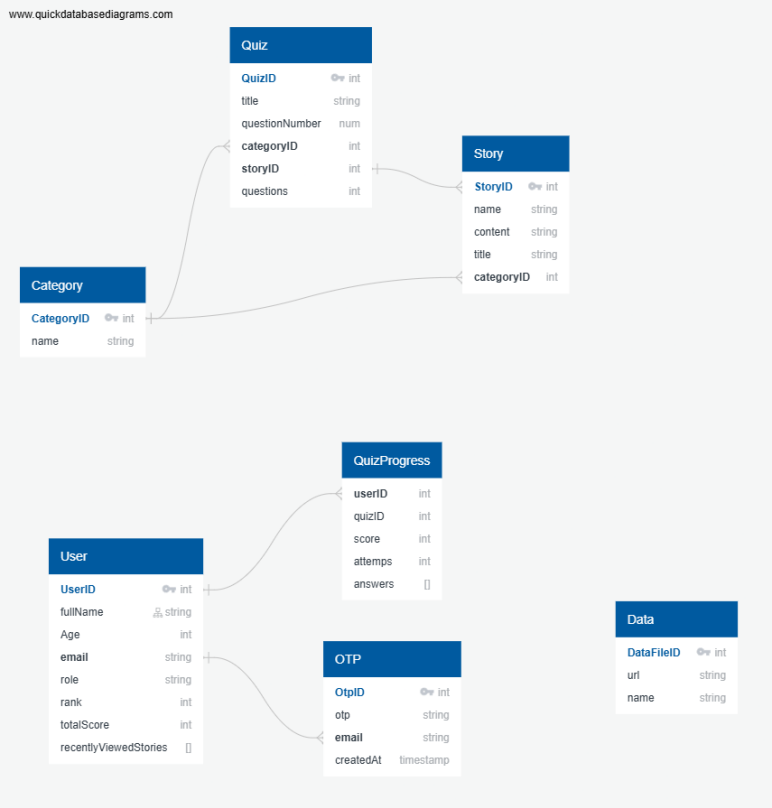
2.1 Data Management & Storage

The backend handles structured data storage using **MongoDB**, with **Mongoose** as the ODM (Object Data Modeling) library to manage data efficiently. Mongoose provides solutions for maintaining data consistency, including **data embedding and referencing**, which makes it a great choice for our application. By embedding related data within documents, we can optimize read performance, while referencing allows for flexibility and scalability when handling larger datasets.

Our database is designed to accommodate the following key collections:

- **Users**: Stores user profiles, authentication credentials, and learning progress. It includes user details such as name, email, age, role, rank, total score, and recently viewed stories.

- **Categories** Organizes stories into structured sections, grouping them based on predefined classifications to enhance navigation and learning.
- **Stories**: Contains historical narratives and educational content. Each story is associated with a category, these stories are not only informative but also engaging and attractive for kids, designed to capture their interest and enhance their learning experience.
- **Quiz Progress:** Holds questions linked to specific story, each quiz containing the questions and the correct answer of it.
- **OTP:** Manages the one-time passwords (OTPs) for authentication purposes. It stores OTPs linked to user emails and timestamps to handle secure authentication and password resets
- **Data Collection**: Contains the learning content of the platform, ensuring all educational materials are readily accessible.



Using Mongoose's schema modeling, validation, and middleware support, our application ensures structured and efficient data management while maintaining flexibility for future expansion.

2.2 User Authentication & Authorization

Secure user authentication is implemented using **JWT (JSON Web Tokens)**. The authentication system includes:

- **User Registration & Login**: Supports email-password authentication with password hashing (bcrypt).
- **Role-Based Access Control (RBAC)**: Differentiates access levels for **users and administrators**.
- **Token-Based Authentication**: Uses **access tokens** to manage session expiration and security.
- **Token Management & Updates**: If a user updates their profile information, such as their email or age, a new token is issued to reflect these changes and ensure continued security.
- **Account Deletion & Token Invalidation**: If a user deletes their account, all associated tokens are immediately invalidated, preventing any further access using previously issued tokens. This ensures that once an account is removed, no authentication attempts can be made using old credentials.

2.3 Content Delivery & API Management

The backend provides structured **RESTful APIs** to serve content dynamically. This includes:

- **Story Retrieval**: Fetching stories categorized by topics.
- **Quiz Management**: Fetching quizzes, processing answers, and calculating scores.
- **Progress Tracking**: Allowing users to resume their learning sessions from where they left off.

All API responses follow a **standardized JSON format** to ensure consistency and ease of integration.

2.4 User Progress Tracking

Progress tracking is implemented using dedicated collections:

- **Quiz Progress (quizProgress)**:
    - Since **quiz progress contributes to rankings and achievements**, it's worth mentioning how it's used in the **leaderboard**.
- **Story Progress (storyProgress)**:
    - in the **ERD**, there is no explicit storyProgress collection. Instead, **recentlyViewedStories** is stored in the **User** collection.

This enables personalized learning experiences, allowing users to track their achievements.

2.5 Interactive Features & AI Integration

To enhance engagement, the backend integrates AI-powered voice and text-based Q&A using Google Generative AI. Key features include:

- **Chatbot Assistance**: Provides real-time responses to historical questions based on data entered by educators.
- **Voice Commands**: Enables users to interact with the app using voice input, with responses delivered through voice output.

This AI-driven approach improves accessibility and learning retention, ensuring that interactions are tailored to the educational content provided.

This part of Application is built using **FastAPI**, a high-performance web framework for Python, ensuring efficient request handling and real-time responses. The FastAPI backend powers the model's ability to process educator-uploaded data files, improving the accuracy of responses. It provides a robust API for configuring and interacting with the AI model, making it easily accessible for frontend integration.

3. API Endpoints

Base URL: https://gp-back-end-beige.vercel.app/api/auth

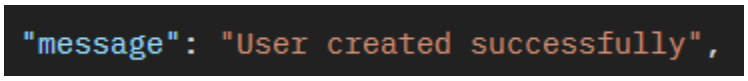
3.1 Authentication APIs

```
{
    "fullName":"koko mohamed",
    "email":"jemob48449@eluxeer.com",
    "password":"Koko123#",
    "confirmPassword":"Koko123#",
    "age":10
}
```

User Registration (Sign Up)

- **URL:** /signUp
- **Method:** POST
- **Description:** Registers a new user.
- **Request Body:**

- **Request Body:**



Status Codes:

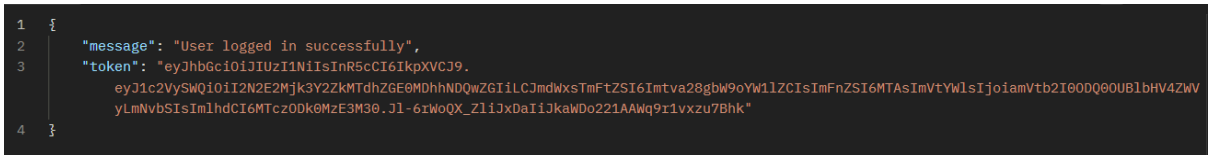
- **201** Created: User registered successfully.
- **400** Bad Request: Missing or invalid input.
- **409** Conflict: Email already exists.

User Login (Sign in)

- **URL:** /signIn
- **Method:** POST
- **Description:** Authenticates an existing user and returns a JWT token.
- **Request Body:**



-
- **Response Body:**



Status Codes:

· 200 OK: Login successful.

· 401 Unauthorized: Invalid credentials.

Send OTP

- **URL:** / sendOTP
- **Method:** POST
- **Description:** Sends a one-time password (OTP) to the user's email for verification.
- **Request Body:**

```
{
    "email":"jemob48449@eluxeer.com"
}
```

- **Response Body:**

```
{
    "message": "OTP sent successfully"
}
```

Status Codes:

- **200** OK**:** OTP sent**.**
- **404** Not Found: Email not registered

Verify OTP

- **URL:** / verifyOTP
- **Method:** POST
- **Description:** Verifies the OTP entered by the user
- **Request Body:**

```
{
    "otp": "2776"
}
```

- **Response Body:**

```
{
    "message": "OTP verified successfully"
}
```

Status Codes:

- **200** OK**:** OTP is valid**.**
- **400** bad requests: Incorrect or expired OTP

Reset password

- **URL:** / resetPassword
- **Method:** PUT
- **Description:** Allows users to reset their password after verifying OTP.
- **Request Body:**

```
{
    "email":"diyax27067@perceint.com",
    "password":"Ahmed123#",
    "confirmPassword":"Ahmed123#"
}
```

- **Response Body:**

```
{
    "message": "password reset successfully"
}
```

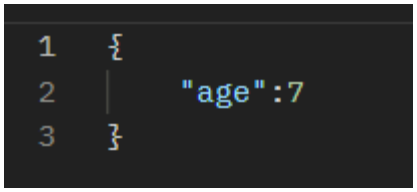
Status Codes:

- **200** OK**:** Password reset successfully**.**

- **400** bad requests: Invalid email or password format

Update Logged-in User Data

- **URL:** / updateUserData
- **Method:** PUT
- **Description:** Updates user profile information.
- **Headers:** Token
- **Request Body:**



- **Response Body:**



Status Codes:

- **200** OK**:** User updated successfully**.**
- **401** Unauthorized: Token missing or invalid.

Change Logged-in User Password

- **URL:** / updatePassword
- **Method:** PUT
- **Description:** Allows authenticated users to update their password.
- **Headers:** Token
- **Request Body:**

```
{
    "currentPassword":"Ahmed123#",
    "newPassword":"Koko123#",
    "confirmPassword":"Koko123#"
}
```

- **Response Body:**

```
{
    "message": "password updated successfully"
}
```

Status Codes:

- **200** OK**:** Password updated successfully.
- **401** Unauthorized: Invalid token or incorrect current password.

Delete Account

- **URL:** / DeleteAccount
- **Method:** DELETE
- **Description:** Deletes the user's account permanently.
- **Headers:** Token
- **Request Body:**

```
{
    "password":"Koko123#"
}
```

- **Response Body:**

```
{
    "message":"Account deleted successfully"
}
```

Status Codes:

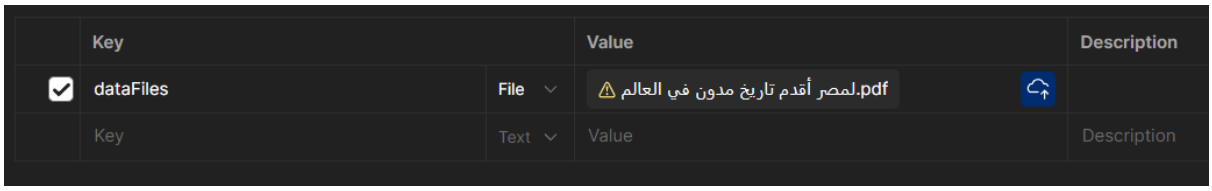
- **200** OK**:** User deleted
- **401** Unauthorized: Invalid token or incorrect current password

Base URL: https://gp-back-end-beige.vercel.app/api/data

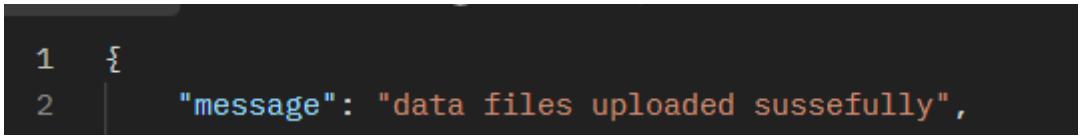
3.2 Data APIs

Upload File

- **URL:** /uploadData
- **Method:** POST
- **Description:** Registers a new user.
- **Request Body:**



- **Request Body:**



Status Codes:

- **201** OK: File uploaded successfully.
- **400** Bad Request: Missing file or invalid format
- **413** Payload Too Large: File exceeds maximum allowed size

Base URL: https://gpfirst.vercel.app/

Configure Gemini history

- **URL:** /
- **Method:** get
- **Description:** configure gemini history with the data files the educator uploaded to give accurate answer

- **Response Body:**



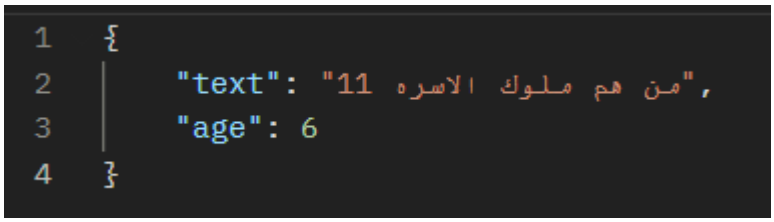
```
{
    "message": {
        "message": "Hello from our model!"
    }
}
```

Status Codes:

- **200** OK: Model learned successfully.

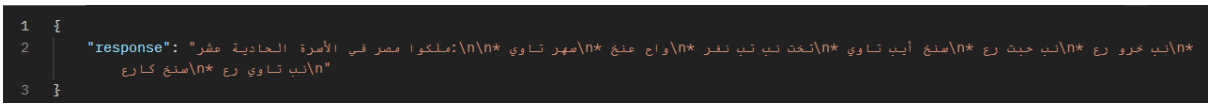
Get Answer

- **URL:** /
- **Method:** post
- **Description:** get answer to your question from the custom data upload and also the answer is suitable for child age
- **Request Body:**



```
{
    "text": "من هم ملوك الاسره 11",
    "age": 6
}
```

- **Response Body:**



```
{
    "response": "ملكوا مصر في الأسرة الحادية عشر:\n\n* سهر تاوي\n* واح عنخ\n* تخت نب تب نفر\n* عنخ أيب تاوي\n* نب حبت رع\n* نب خرو رع\n* نب تاوي رع\n* صنخ كارع\n"
}
```

Status Codes:

- **200** OK: Answer returned successfully

4. Deployment & Infrastructure

4.1 Database: MongoDB Atlas

The database for this application is deployed on **MongoDB Atlas**, a cloud-based NoSQL database service that provides high availability, scalability, and security. The database is hosted under the **gp** folder and is managed within **Cluster 1**. The primary database used in this project is named **test**, where all collections related to users, quizzes, stories, and progress tracking are stored. The connection to MongoDB Atlas is securely managed using environment variables to ensure data protection. Authentication is configured with IP whitelisting and username/password credentials to enhance security. The backend connects to the database using **Mongoose**, a powerful Object Data Modeling (ODM) library for MongoDB, allowing efficient data handling and seamless integration with the application.

The **NoSQL schema design** allows flexible and scalable data retrieval while maintaining data integrity.

4.2 Vercel

To ensure seamless backend integration with the frontend team, the project is deployed on **Vercel**, a cloud platform that provides serverless hosting with automatic scaling. The deployment is divided into two separate services for better modularity and performance. The first deployment, named **gpFirst**, is dedicated to handling the **machine learning model**, which is responsible for learning processes and answering user questions. The second deployment, named **gp**, manages the **core backend functionalities**, including **authentication, data uploading, OTP verification, and API routes**. These structured deployments allow the frontend team to interact with the backend through well-defined API endpoints, making collaboration efficient. The APIs are accessible via Vercel's generated URLs, ensuring real-time updates and easy testing. Environment variables are securely stored in Vercel, protecting sensitive credentials such as database connection strings and API keys. By leveraging Vercel's integration with GitHub, every update pushed to the repository triggers an automatic deployment, ensuring that the backend remains in sync with ongoing frontend development.

The **NoSQL schema design** allows flexible and scalable data retrieval while maintaining data integrity.

4.3 Github

The project's source code is managed using **GitHub**, which serves as the version control system to track changes, collaborate efficiently, and maintain code quality. To ensure a structured development process, every new feature or update is first implemented in a separate **feature branch**, preventing disruptions to the main codebase. Once the development and testing of a feature are complete, the branch is merged into the main branch through a **pull request (PR)**, ensuring proper review and maintaining stability. This workflow allows for seamless collaboration with the frontend team and ensures that the backend remains scalable and maintainable. GitHub is also integrated with **Vercel**, enabling automatic deployments whenever changes are merged, keeping the live backend up to date without manual intervention. By following this branching strategy, the project maintains a clean and organized code structure while enabling continuous improvement and feature expansion

The **NoSQL schema design** allows flexible and scalable data retrieval while maintaining data integrity.

5. Deployment & Infrastructure

In the upcoming development phase, the focus will be on expanding and refining key backend functionalities, particularly around **content generation, user engagement, and API enhancements**. The following improvements are planned:

5.1 Categories & Stories Endpoints

· Develop and optimize **category and story APIs** to ensure seamless content retrieval and organization.

· Implement **AI-driven story generation**, where content is dynamically created based on predefined themes and topics.

5.2 Quizzes API Enhancements

· Automate **AI-powered quiz generation**, ensuring that quizzes are dynamically created from story content.

· Ensure **questions vary in difficulty** to accommodate different user levels and effectively assess comprehension of the story.

· Improve **quiz validation mechanisms** to ensure content accuracy and engagement

5.3 Scoring mechanism

Users will earn points based on **quiz performance**, with scores calculated using:

- **Correct answers**: Higher points awarded for correct responses.
- **Difficulty level**: More points for answering harder questions correctly.
- **First attempt bonus**: Extra points for getting the correct answer on the first try.

5.4 Leaderboard Updates

The **leaderboard** will be dynamically updated based on:

- **Total accumulated points**: Users with higher scores rank higher.

5.5 Custom AI Model APIs

· Develop **custom AI models** to enhance personalized learning experiences.

· Improve **AI-Model learning** by reducing the time we need make the model ready in

11. Frontend Part

1. Intro:

In this app, we've crafted a dynamic and responsive user interface using Flutter, an open-source framework powered by the Dart programming language. Flutter allows developers to build high-performance, cross-platform applications (Android, iOS, Web, and Desktop) from a single codebase. Its flexibility and speed make it an ideal choice for delivering seamless user experiences efficiently.

2. Technologies and Tools:

- **Framework**: Flutter
- **Language**: Dart
- **Libraries and Packages**:

    i. Flutter/material: Handles UI components and imports essential widgets.
    ii. SpeechToText: Converts speech to text for chatbot interactions.
    iii. Flutter_tts: Manages text-to-speech functionality for responding to user queries.
    iv. file_picker: Allows users to pick file from device storage.

v. mimi: Verifies the file type to ensure it's a pdf or not.
vi. http: Manages API requests and handles server communication.
vii. modal_progress_hud_nsn: Display loading indicator in most of screens.
viii. dart_jsonwebtoken: Decode user token for secure authentication.
ix. 'permission_handler': Handles runtime permissions for microphone access.
x. shared_preferences: securely stores authentication tokens and manages local data flow within the app.

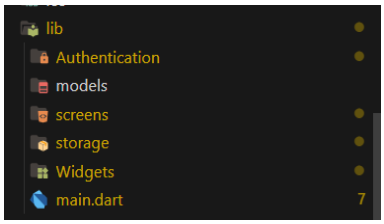
- **Additional tools**: Github (version control), Figma (UI/UX design).

3. Project Structure:

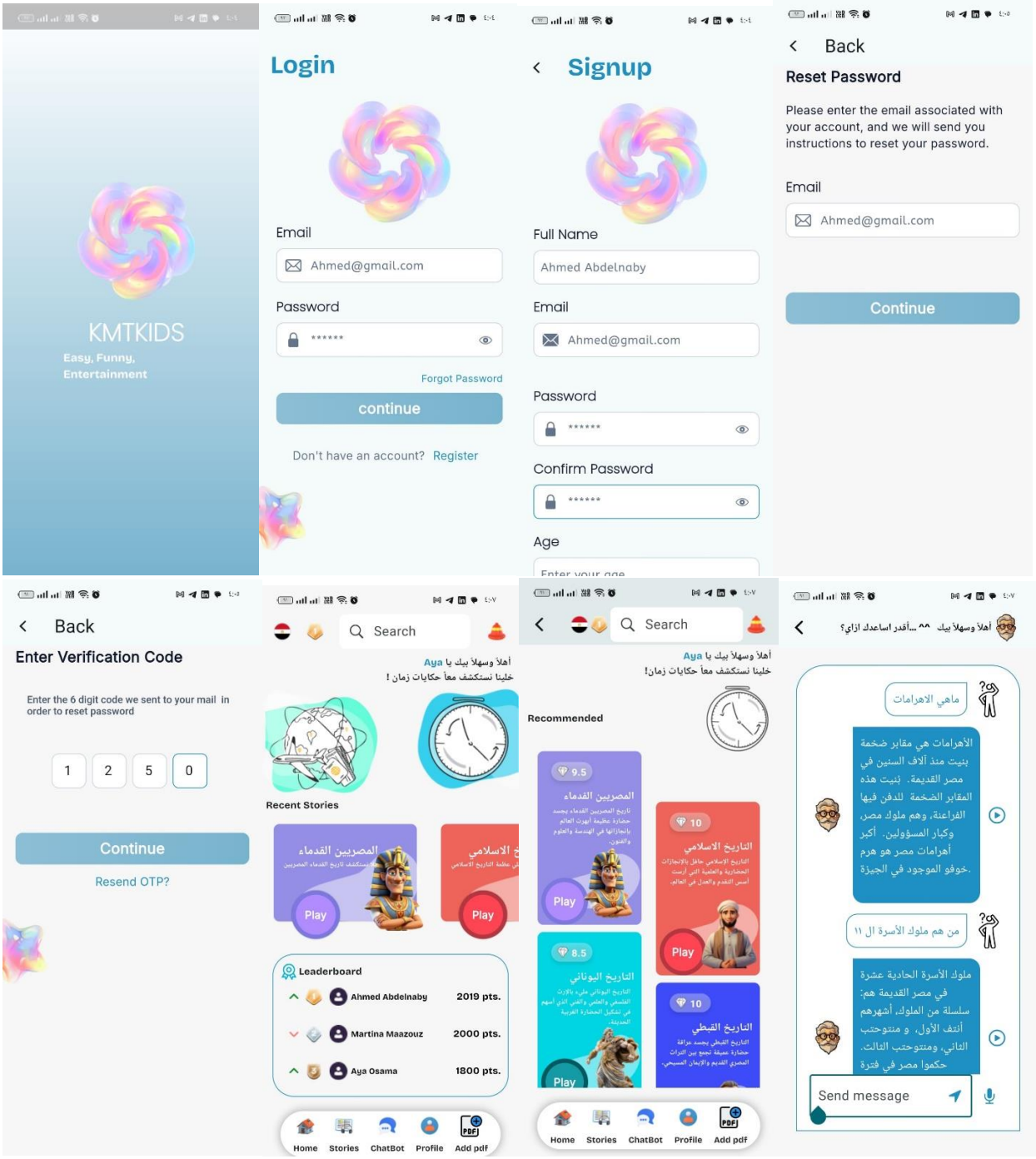
- **Design Pattern**: MVC
    1. Models: Manages the app data, logic and API communication.
    2. View: Handles the user interface and presentation layer
    3. Controller : Acts as a bride between the model and view (manages user input and app behavior)

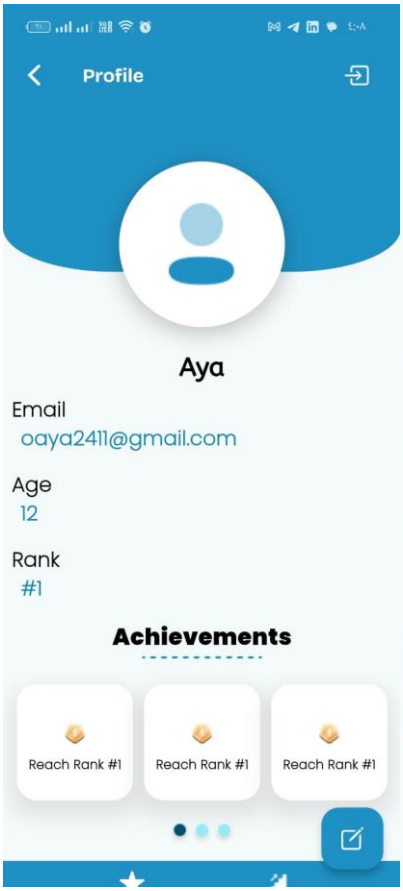
- **Project design:**

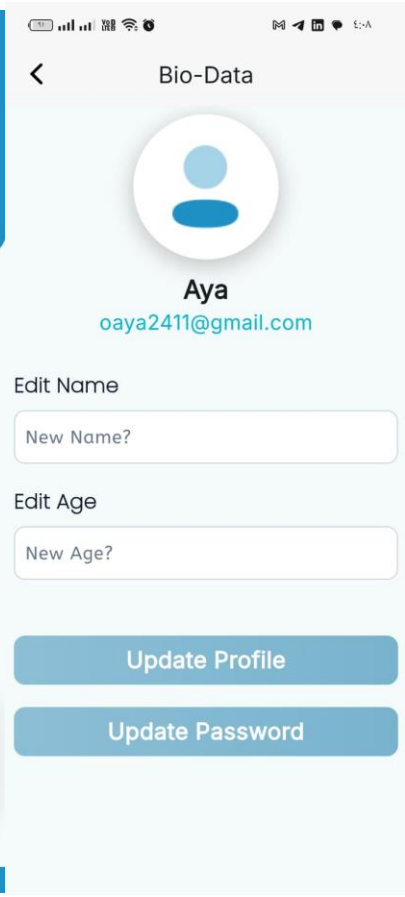
→ Folder Structure:

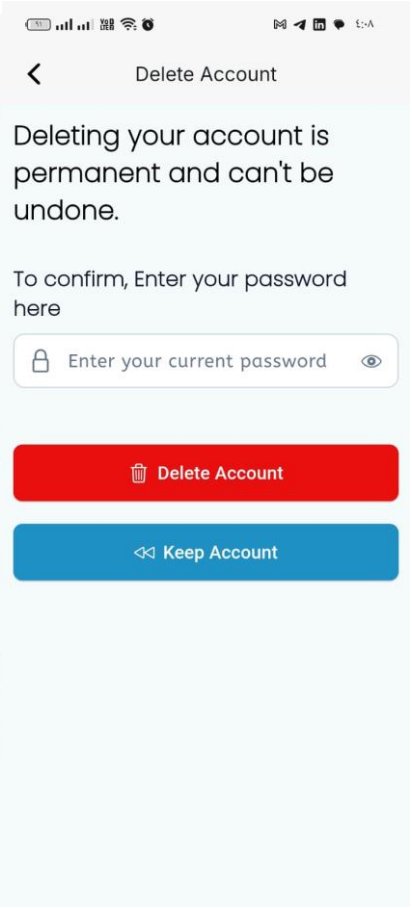


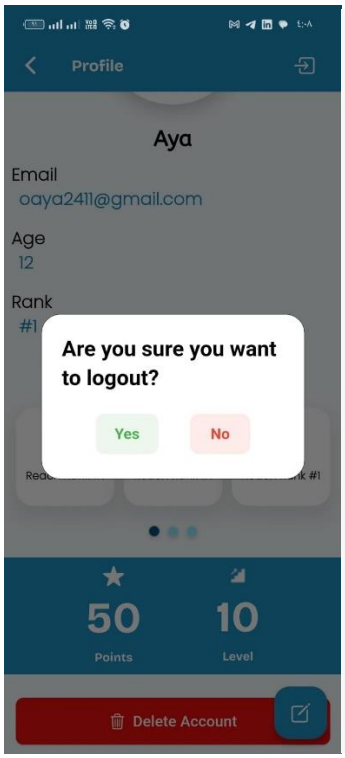
→ App pages:

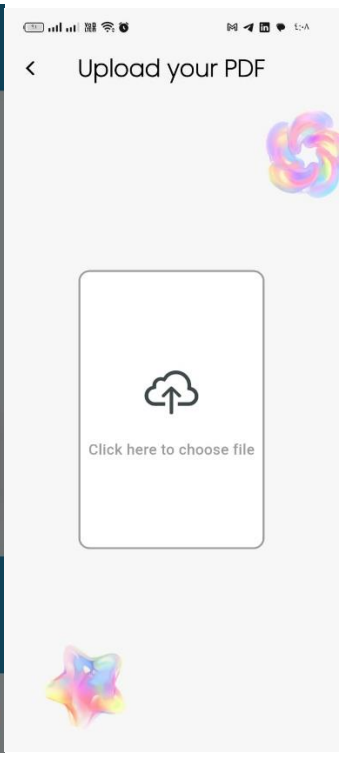

KMTKIDS
Easy, Funny,
Entertainment
Login
Email
Ahmed@gmail.com
Password
******
Forgot Password
continue
Don't have an account? Register
Signup
Full Name
Ahmed Abdelnaby
Email
Ahmed@gmail.com
Password
******
Confirm Password
******
Age
Enter your age
Back
Reset Password
Please enter the email associated with your account, and we will send you instructions to reset your password.
Email
Ahmed@gmail.com
Continue
Back
Enter Verification Code
Enter the 6 digit code we sent to your mail in order to reset password
1 2 5 0
Continue
Resend OTP?
Search
أهلا وسهلا بيك يا Aya
خلينا نستكشف معاً حكايات زمان !
Recent Stories
المصريين القدماء
Play
Leaderboard
Ahmed Abdelnaby 2019 pts.
Martina Maazouz 2000 pts.
Aya Osama 1800 pts.
Home
Stories
ChatBot
Profile
Add pdf
Search
أهلا وسهلا بيك يا Aya
خلينا نستكشف معاً حكايات زمان!
Recommended
9.5
المصريين القدماء
Play
10
التاريخ الاسلامي
Play
8.5
التاريخ اليوناني
Play
10
التاريخ القبطي
Home
Stories
ChatBot
Profile
Add pdf
أهلا وسهلا بيك ... أقدر اساعدك ازاي؟
ماهي الاهرامات
الأهرامات هي مقابر ضخمة بنيت منذ آلاف السنين في مصر القديمة. بُنيت هذه المقابر الضخمة للدفن فيها الفراعنة، وهم ملوك مصر، وكبار المسؤولين. أكبر أهرامات مصر هو هرم خوفو الموجود في الجيزة.
من هم ملوك الأسرة ال ١١
ملوك الأسرة الحادية عشرة في مصر القديمة هم: سلسلة من الملوك، أشهرهم أنتف الأول، و منتوحتب الثاني، ومنتوحتب الثالث. حكموا مصر في فترة
Send message


Profile
Aya
Email
oaya2411@gmail.com
Age
12
Rank
#1
Achievements
Reach Rank #1
Reach Rank #1
Reach Rank #1


Bio-Data
Aya
oaya2411@gmail.com
Edit Name
New Name?
Edit Age
New Age?
Update Profile
Update Password


Delete Account
Deleting your account is permanent and can't be undone.
To confirm, Enter your password here
Enter your current password
Delete Account
Keep Account


Profile
Aya
Email
oaya2411@gmail.com
Age
12
Rank
#1
Are you sure you want to logout?
Yes
No
50
Points
10
Level
Delete Account


Upload your PDF
Click here to choose file

Let us know more details about each page:

- **Welcome Screen:**

  This screen represents the welcome screen of the app, displaying the logo and a brief description of the app. It performs a token check to navigate to either the `***Home***` screen if the token is valid or the `***Login***` screen if the token is missing or invalid.

- The main functionality for this page is:

I. **Decode user token**: it decodes the JWT token to retrieve user data.
II. **Navigation**: After a 3 seconds(for ending animation for the animated logo) the screen check if the token is exist and valid → navigate it to Home screen
If it isn't a valid token → navigate it to Login screen.

- Libraries and packages used:

I. 'dart_jsonweb_token'.
II. 'flutter/matrial.dart'.

- Components (widgets) Explanation:

1. **Scaffold Widget**: it's the root widget in all screens and has the main structure for entire screen.

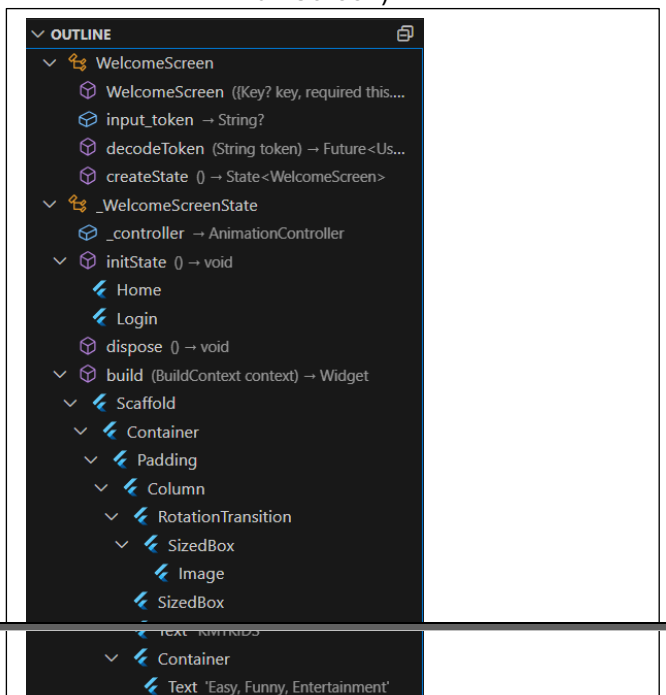
2. **Container Widget**: it's used to create a gradient screen, used in it **padding** Widget to handle the content padding.

3. **Rotation Transition Widget**: it make a rotation animation effect for our logo.

4. **Text Widget**: Display text to the screens.

5. **Sized Box:** Used it to make safe spaces between widgets(and this is in all Screen).

- Widget tree:



- **Login Screen:**
  Screen to login to your profile, contains text fields to enter email and password. It verifies the correctness of the input data then send it to backend API to verify email if it's correct data.

  - **The main functionality**:
    I. **Form Validation**: validate the input data(email and password) by using regex.
    II. **API Request**: make request to ensure that the input data is correct and matches the data in db.
    III. **Handling Loading State**: make the screen dynamically change the state.
    IV. **Token Storage**: store the valid tokens in our cache Memory using function of 'Cashmemory.insertToCashe' and store it for using later.
    V. **Navigation**: navigate to 'Home' screen in case that the data is correct and in case of wrong data, the app display the problem in specific way.
    VI. **Navigate to other screens:** the Register screen and Forgot Password screen.

- **Libraries and Packages:**
  I. 'flutter/material.dart'
  II. 'http'
  III. 'modal_progress_hud_nsn'
  IV. 'Cupertino_icons'

  - **Components (widgets) Explanation:**

    1. **Scaffold**: Provide the basic structure for the page. It contains of appBar and Body. App bar is customized according to page design.

    2. **ModalProgressHuD**: Display loading spinner while waiting for the API response and this enhances the user experience during background processing.

    3. **Form**: contains the fields of email and password. It has a validator which ensure about the validation of the entire data using **Global Key** which link the form with its validation state.

    4. **Custom TextField**: It's a custom text field which we made(will be explained in details late). Used for both email and password fields.

    5. **GestureDetector:** Listens for tap event and used to navigate to reset password screen on tap 'Forgot Password' section in the.

    6. **ButtonContainer:** custom button we made to be with specified design.

7. **TextButton:** Allowing users to navigate to register screen if they don't have email.

8. **Positioned:** Display a star image with custom positioning and rotation.

9. **Row & Column:** Used to organize and align elements horizontally (Row) and vertically (Column) within the page.

- **State Management:**

❖ **Variables**:
> **Email, password**: store user input.
> **isLoading**: indicating the loading state during API communication.
> **URL:** determine the link of API.

❖ **Form Validation**:
> This is done using regular expressions (regex).
> We define two functions: one for email validation to ensure the correct email format, and one for password validation to ensure it includes uppercase and lowercase letters, numbers, and at least one special character.

❖ **Send Data function:**

> The function responsible for sending data to the API using http.post.

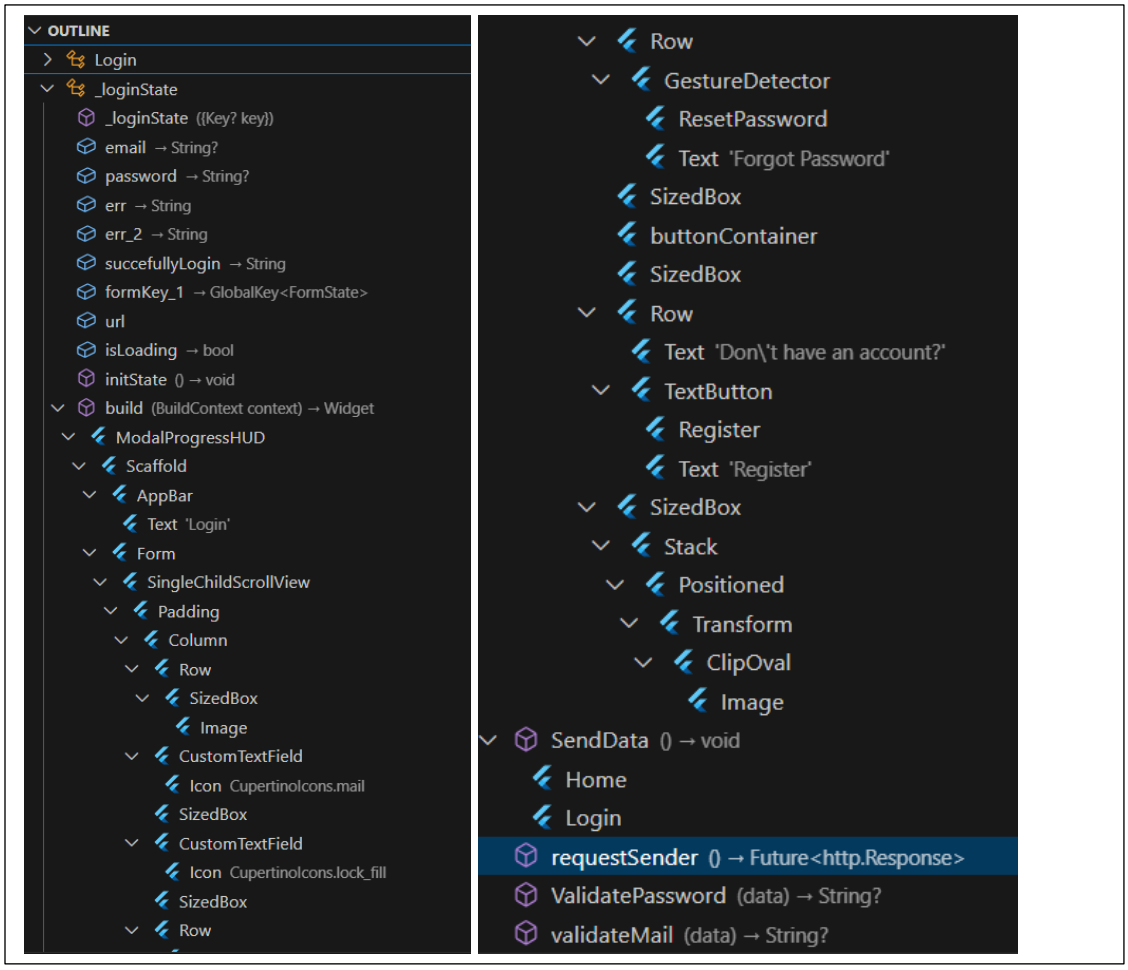
> If the inputs are valid, the request is sent to the backend.

> On success, the token is stored using CacheMemory, display SuccessMessage and the user is redirected to the Home screen.
> On failure, an error message is displayed using FailedMessage**.**

❖ **Request Sender function**:

> Contains the body of the request and send the request via post ,request

- **Widget Tree:**



➢ **Register Screen:**
The register screen allows new users to create an account by providing their full name, email, password and age.

- **The main functionality**:
    I. **Form validation**: Ensures all fields are filled correctly using custom validation functions for name, email, password, confirmation password, and age.
    II. **Data Submission**: Sends the validated data to the backend API for user registration.
    III. **Error Handling:** Displays error messages if the email already exists or if validation(for any of the input data) fails.
    IV. **Loading Indicator:** Shows a loading spinner while the registration process is done.

V. **Navigation**: Redirects to the confirmation screen upon successful registration or display failed message with the occurred problem to handle it.

- **Libraries and Packages**:
    I. ‘flutter/material.dart’
    II. ‘http’
    III. ‘modal_progress_hud_nsn’
    IV. ‘Cupertino_icons’

- **Components (widgets) Explanation**:
    1. **Scaffold**: Provide the basic structure for the page. It contains AppBar and Body. AppBar is customized according to page design.

    2. **ModalProgressHuD**: Display loading spinner while waiting for the API response and this enhances the user experience during background processing.

    3. **Form**: contains the fields of email and password. It has a validator which ensure about the validation of the entire data using **Global Key** which link the form with its validation state.

    4. **SignleChildScrollView**: Enables scrolling when the content on it exceeds the screen’s visible area.

    5. **CustomTextField**: It’s a custom text field which we made. Used for both email and password fields.

    6. **buttonContainer**: it’s a custom button we made(will be explained later in details). We use it to send request after validation data in this page.

    7. **TextButton:** used to navigate to login page as the user might make account with the same email before.

    8. **Positioned:** Display a star image with custom positioning and rotation.

    9. **Row & Column:** using to organize and align elements horizontally and vertically.

    10. **Other widgets**: Text widget, Padding widget and sizedBox.

- **Additional Functions:**
  - ❖ **Form Validation**:

> This is done using regular expressions (regex).

> We define 4 functions: ValidateName, ValidateEmail, ValidateAge, ValidatePassword, ConfirmPass

  - ❖ **Send Data function:**

> The function responsible for sending data to the API using http.post.

> If the inputs are valid, the request is sent to the backend.

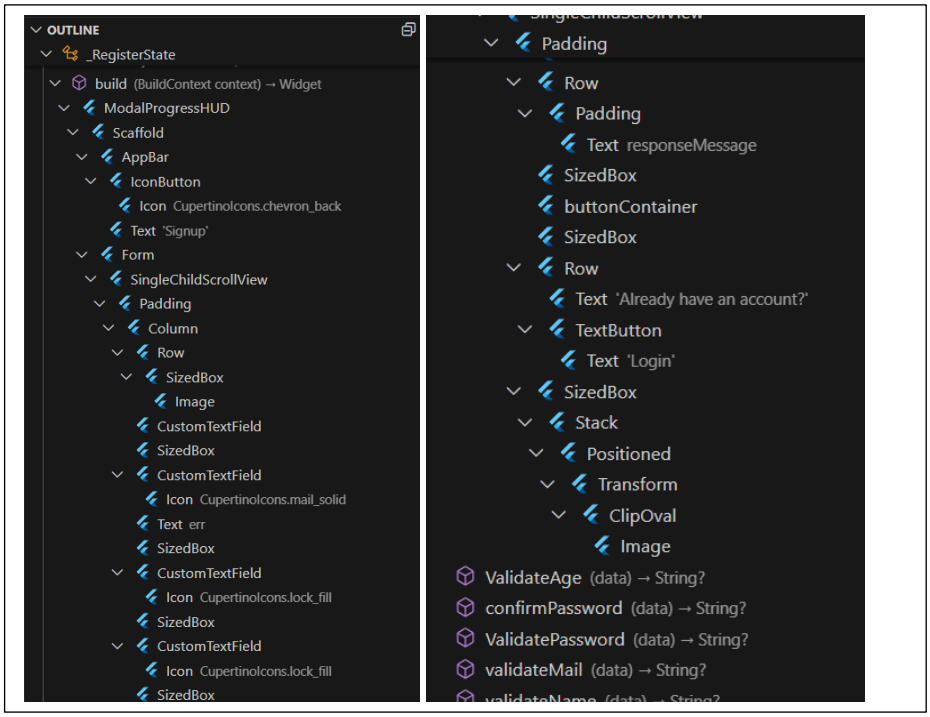
> On success, display success message then push confirmation screen.

> On failure, an error message is displayed using Fail Message**.**

  - ❖ **SendInternalData function**:

> Contains the body of the request and send the request via post request.

- **Widget Tree:**



➢ **Confirmation screen:**
The screen only confirm user that his mail is done successfully and he must login with it to ensure that the email is exist.

- **Functionality**:

I. Confirm that mail user is done.
II. Navigate him to the login screen to login with his new account.

- **Libraries and packeges:**
  I. ‘flutter/matrial.dart’

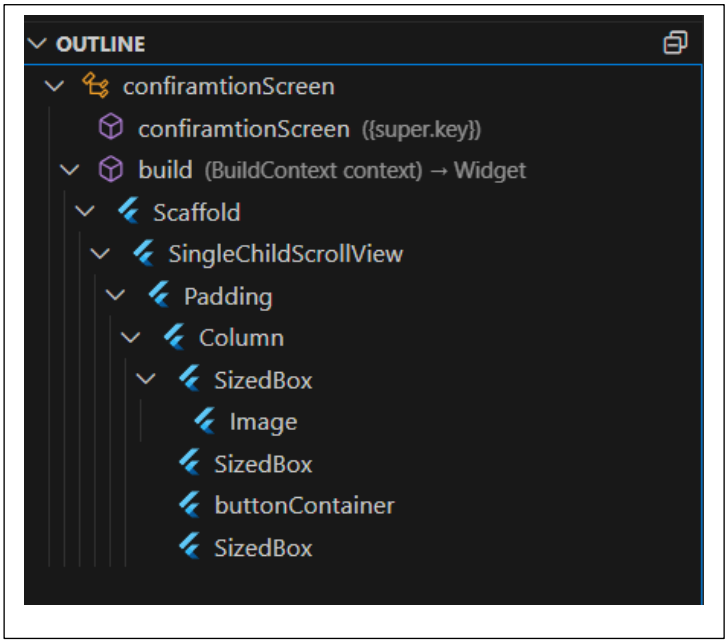
- **Component (widgets) Explanation:**
  1. **Scaffold**: Provide the basic structure for the page. It contains AppBar and Body. AppBar is customized according to page design.

  2. **SignleChildScrollView**: Enables scrolling when the content on it exceeds the screen’s visible area.

  3. **Image**: used to display asset image in the screen.

  4. **buttonContainer**: it’s a custom button we made(will be explained later in details). We use it to send request after validation data in this page.

- **Widgets Tree:**



➢ **Forgot Password Screens:**

This document covers the Password Recovery Flow, which consists of three interconnected screens: Forgot Password, OTP Verification, and Reset Password. These screens work together to help users securely recover their accounts in case they forget their passwords.

- **Sequence Flow:**

  The password recovery process follows this sequence:

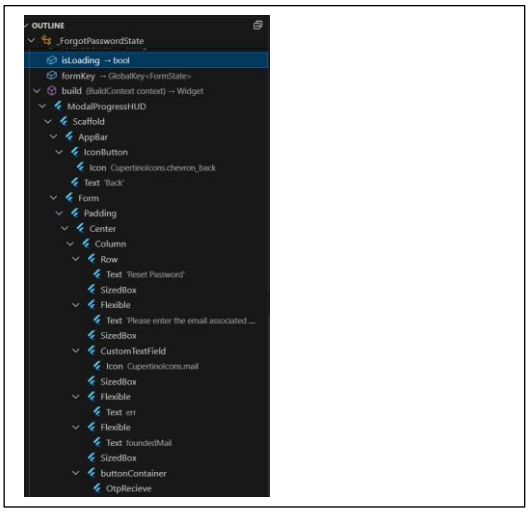
  1. **Forgot Password**: The user enters their registered email to request a password reset.

  2. **OTP Verification**: The user receives a One-Time Password (OTP) via email and enters it to verify their identity.

  3. **Reset Password**: Upon successful OTP verification, the user sets a new password.

- **Individual Pages Documentation:**

  1. **Forgot Password Screen:**
     - **Functionality**:
       - Collects the user's email address.
       - Send a password reset request to the backend.
       - Display success or fail messages based on the response.
     - **Widgets Explanation**:
       - CustomTextField: For email input with validation.
       - buttonContainer: A button to submit the email.
       - Text: Displays success/error messages.
     - **Additional Functions**:
       - Manages email input state, loading state, and error messages.
       - Send API post request to ensure about the existence of the mail.
     - **Widget Tree**:



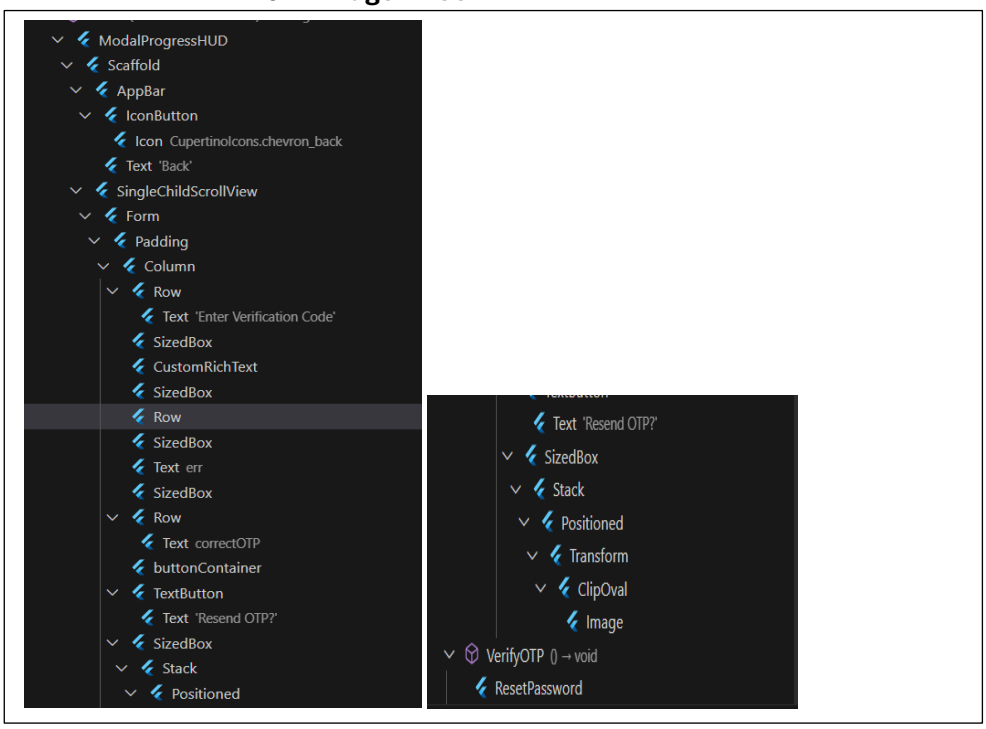
  2. **OTP Screen:**
     - **Functionality**:

- Accepts the OTP sent to the user's email.
- Validates the OTP with the backend.
- Navigates to the Reset Password page upon successful verification.

- **Widgets Explanation**:
  - CustomTextField: For OTP input.
  - buttonContainer: For OTP submission.
  - Text: To display validation errors.
- **Additional Functions:**
  - Manages OTP input and validate it using validator to ensure

    that given data is only numbers.
  - verifyOTP Function: this function send API post request that take OTP and show me if it true or not.

- **Widget Tree**:



3. **ResetPassword Screen:**
   - **Functionality:**

     - Allows the user to enter their registered email, set a new password, and confirm it.

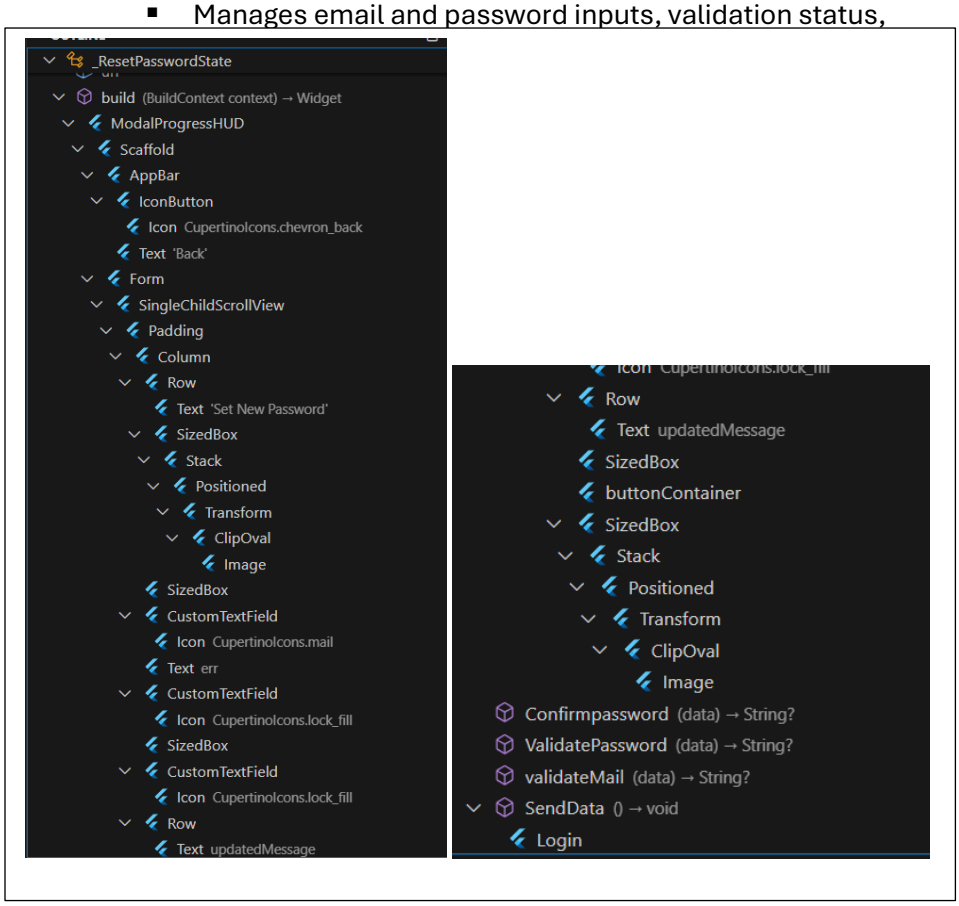
- Validates email and password inputs.
- Submits the new password to the backend for updating the user's account.
- Displays success messages and redirects to the login page.

- **Widgets Explanation:**

  - **CustomTextField:** For email, new password, and confirmation input with validation.
  - **buttonContainer:** To submit the new password.
  - **Text:** For error/success messages.

- **Additional Function:**

  - Manages email and password inputs, validation status,



➢ **Chatbot Screen:**

This screen represents the main function for our app as we handle a chatbot that facilitates both text and voice interactions with a chatbot API which connected to AI model. It integrates speech-to-text capabilities, allowing users to either type or speak

their messages. The screen is designed with a clean UI, supporting message display in both sender and receiver formats.

- **Functionality**:
  i. **Text Messaging**: Users can type their messages into a TextField and send them by tapping the send icon.
  ii. **Voice Recognition**: By tapping the microphone icon, users can start voice recognition to convert speech into text, which is then sent as a message.
  iii. **API communication**: Messages are sent to an external API and the responses are displayed in the chat interface.
  iv. **Permissions Handling**: The app requests microphone permissions for voice input functionality.
  v. **Auto Scrolling:** The chat automatically scrolls to the latest message after sending or receiving a message.
- **Libraries and Packages:**
  i. 'flutter/matrial.dart'
  ii. 'flutter/Cupertino.dart'
  iii. 'speech_to_text'
  iv. 'http'
  v. 'permission handler'
- **Components (widgets) Explanation**:
  1. **Scaffold**: Provide the basic structure for the page. It contains AppBar and Body. AppBar is customized according to page design.
  2. **ChatAppBar**: it's custom widget with a back icon and row with some components.
  3. **SingleChildScrollView**: Allows the chat screen to be scrollable.
  4. **ListView.builder**: Dynamically builds the list of chat messages, differentiating between sender and receiver messages.
  5. **senderBubble & ReceiverBubble**: Custom widgets to display messages from the user and the chatbot, respectively.
  6. **TextField**: Input field for typing messages, with a send icon.
  7. **IconButton**: Toggles voice recognition using microphone and waveform icons.
  8. **ScrollController**: Manages scrolling behavior to keep the latest message in view. It works with ListView.builder to automatically scroll to the bottom whenever a new message is sent or received, ensuring the latest content is always visible to the user.
  9. **Additional Widgets**: Row, column, Image, Expanded

- **Additional Functions**:

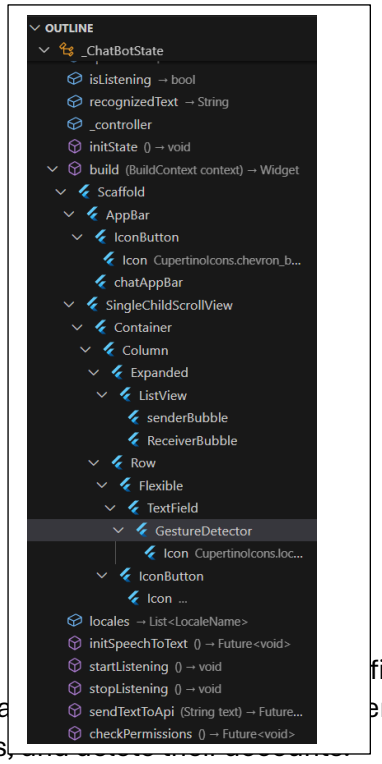
1. **initSpeechToText**(): Initializes speech-to-text services and checks for supported locales.

2. **startListening()** & **stopListening()**: Control the voice recognition process.

3. **sendTextToApi()**: Sends messages to the chatbot API and handles responses.

4. **checkPermissions()**: Requests and checks for microphone permissions.

- **Widget Tree:**



➢ **Profile Screen:**

The ProfilePage is a Flutter-based [illegible] file information, achievements, and account mana[illegible]ers to view and edit their details, check their achievements, [illegible]

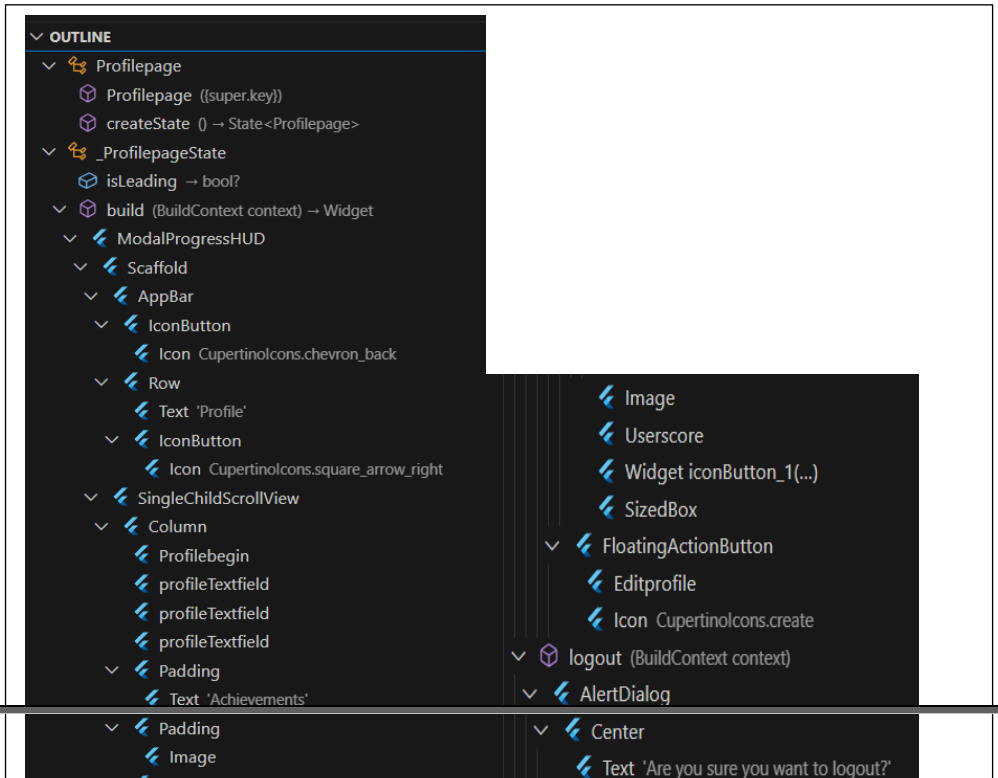
- **Main Functionality:**

i. **View Profile Information**: Displays user data like email, age, and rank using custom text fields.
ii. **Edit Profile**: A floating action button navigates users to the EditProfile screen to modify their details.
iii. **Achievements Display**: Shows user achievements in a horizontal scrollable view.
iv. **Delete Account**: Provides an option to delete the user account with confirmation.
v. **Logout**: Allows users to securely log out, with a confirmation dialog to prevent accidental logouts.
vi. **Navigations**: navigate to update password and delete account screens

- **Libraries and Packages**
    i. ‘flutter/cupertino.dart’
    ii. ‘flutter/material.dart’
    iii. modal_progress_hud_nsn

- **Widgets Explanation**
    1. **Scaffold**: Provide the basic structure for the page. It contains AppBar and Body. AppBar is customized according to page design.
    2. **AppBar**: Contains a back button and a logout button.
    3. **ModalProgressHUD**: Displays a loading spinner during async operations.
    4. **ProfileBegin**: Custom widget displays the header of the profile page.
    5. **ProfileTextField**: Custom widget to display user details like email and age.
    6. **Acheivement**: Custom widget representing individual achievements.
    7. **UserScore**: Custom widget to displays the user's score.
    8. **IconButton**: Custom buttom used for logout and delete account actions.
    9. **FloatingActionButton**: Navigates to the edit profile screen.
    10. **Additional Widgets**: Row, Column, Padding, Image, SignleChildScrollView, sizedBox.
- **Additional Functions**
    1. **logout(BuildContext context):** Displays a confirmation dialog for logging out. If confirmed, calls Cashmemory.logout to handle the logout process.
    2. **Cashmemory.logout(context, token):** Handles the actual logout operation, including clearing user data and navigating to the login screen.

- **Widget Tree:**



➢ **Edit Screen:**

The screen allows users to edit their username and age. It provides validation to the data entered and update it via API put request.

- **Functionality**:
  i. **Edit Name**: User can enter a new name with validation to it.
  ii. **Edit Age:** Users can enter a new age within a specific range.
  iii. **Update profile:** it sends the input data to API and then showing successMessage or FailMessage corresponding to response data.
  iv. **Update password:** Navigate to a new screen to update password and ensure user authenticity before update.

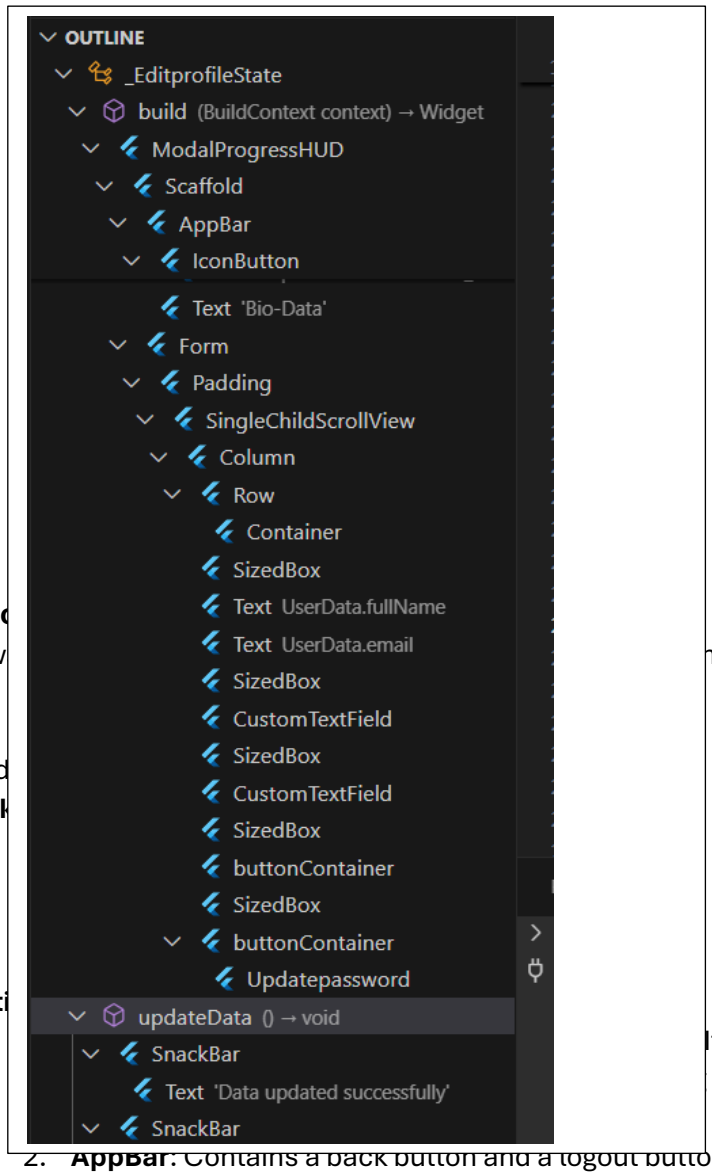
- **Libraries and Packages:**
  i. 'flutter/material.dart'
  ii. 'flutter/Cupertino.dart'
  iii. 'http'
  iv. 'modal_progress_hud_nsn'

- **Components(widgets) Explanation:**
  1. **Scaffold**: Provide the basic structure for the page. It contains AppBar and Body. AppBar is customized according to page design.
  2. **AppBar**: Contains a back button and a logout button.
  3. **ModalProgressHUD**: Displays a loading spinner during async operations.
  4. **Form**: contains the fields of Full Name and Age. It has a validator which ensures the validation of the entire data using Global Key which link the form with its validation state.
  5. **CustomTextField**: Custom textFromField to input the data in it.

6. **ButtonContainer**: Custom button to send data.

7. **Additional widgets**: Row, Column, SingleChildScrollView, SizedBox, iconButton, snackBar, Text.

- **Additional Functions**:
  1. **validateName(String data) validateAge(String data):** Ensures the name starts with letters and may include numbers and that age between 6 & 12.
  2. **updateData():** Sends the updated information to the backend.
  3. **makeRequestMethod:** make the put request and return the response to the above function to use it.
- **Widget Tree:**



➢ **Update Passwo**

The screen allow ne current password.

- **Functionality**: upd
- **Libraries and pack**
  - i.
  - ii.
  - iii.
  - iv.
- **Widgets Explanati**

It contains to page design.

2. **AppBar**: Contains a back button and a logout button.

3. **ModalProgressHUD**: Displays a loading spinner during async operations.

4. **Form**: contains the fields of password. It has a validator which ensures the validation of the entire data using Global Key which links the form with its validation state.

5. **CustomTextField**: Custom textFromField to input the data in it.

6. **ButtonContainer_2:** Another custom button to submit data.

8. **Additional widgets**: Row, Column, SingleChildScrollView, SizedBox, Padding, failedMessage, successMessage, Text.

- **Additional Functions:**
  1. **UpdatePassword:** sends the updates information to the backend.

  2. **sendNewPassword:** sends the put request to api and then return API response to the.

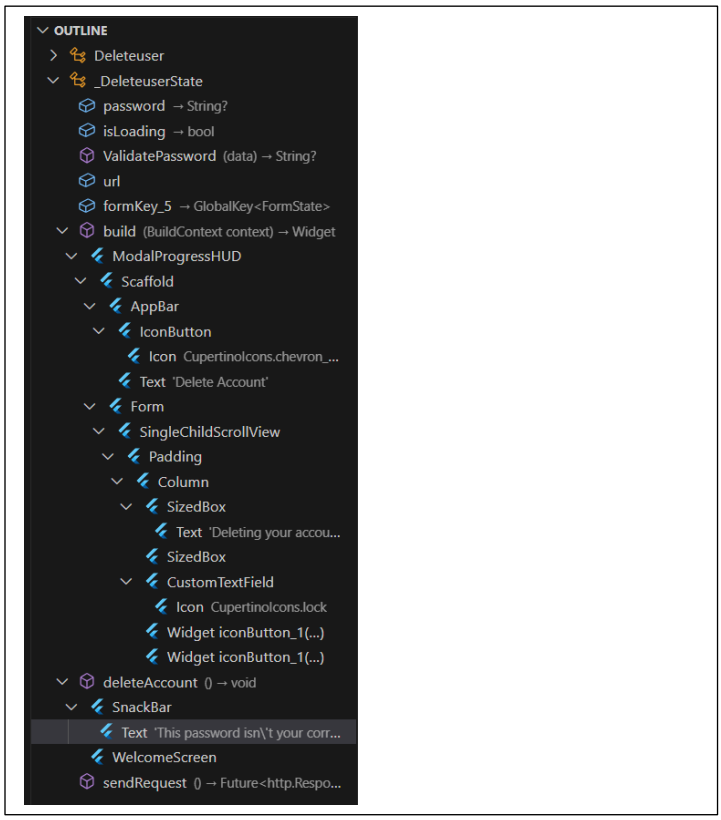
- **Widget Tree:**



  - **Delete Screen:**
    Screen allows us the authority about it.
- **Functionality**: de owner of this account
- **Libraries and Packages**:
  - 'flutter/material.dart'
  - 'flutter/Cupertino.dart'
  - 'http'
  - 'modal_progress_hud_sns'
- **Widgets Explanation:**
  - **Scaffold**: Provide the basic structure for the page. It contains AppBar and Body. AppBar is customized according to page design.
  - **AppBar:** Contains a back button and a logout button.

- **Form**: contains password fiels. It validates the entire data using Global Key which links the form with its validation state.
- **CustomTextField:** Custom text field to enter password in.
- **Icon_button_1:** Custom button with icon on it to submit data.
- **Additional Widgets:** Row, Column, padding, Text, sizedBox, ModalProgressHUD, snackBar, SingleChildScrollView.

- **Additional Functions:**
  - **deleteAccount**: sends API del request to backend.
  - **sendRequest**: make a request here and return the response to function above.
- **Widget Tree**:



➢ **Story Page:**

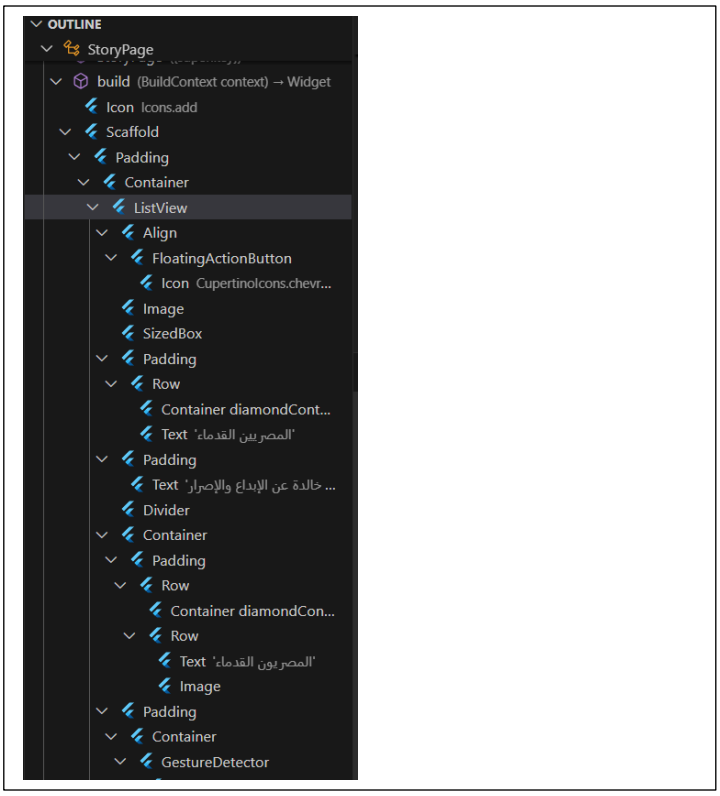
The screen designed to present historical stories interactively. It displays a featured image, story details, and options for user engagement like playing audio and starting quiz.

- **Functionality:**
  - **Back Navigation:** A floating action button allows users to return to the previous screen.
  - **Display Image:** The main story image is showcased at the top.
  - **Story Content:** Provides a detailed historical narrative.
  - **Rating Display:** Displays a rating using a diamond container.
  - **Play Button:** Allows users to listen to an audio narration of the story.
  - **Quiz Start Button:** Enables users to start a quiz based on the story.

- **Libraries and Packages**: flutter/material.dart', 'flutter/Cupertino.dart'
- **Widgets Explanation:**

  - **Scaffold:** The basic widge.
  - **FloatingActionButton**: Provides navigation back to the previous screen.
  - **diamondContainer**: Custom widget displaying the rating score with an icon.
  - **Divider**: Adds a visual separator between sections.
  - **Additional Widgets:** Container, Image, Text, SizedBox.

- **Widget Tree:**



➢ **Home Screen:**

- **The main functionality**:

  I. **Api request :** Fetches data from the backend API to display real-time updates on the home screen.

  II. **Dynamic UI Updates:** The screen updates dynamically based on the API response and user interactions.

  III. **User Greeting:** Displays a personalized greeting using the stored user information.

  IV. **Navigation:** Allows users to navigate to different screens such as StoryPage, ChatBot, UploadFile, and ProfilePage.

V. **Search Functionality:** Provides a search bar for users to find relevant content quickly.

VI. **Leaderboard Display:** Shows a ranking of top users based on their points and performance.

- **Libraries and Packages:**

I. 'flutter/material.dart'

II. 'http'

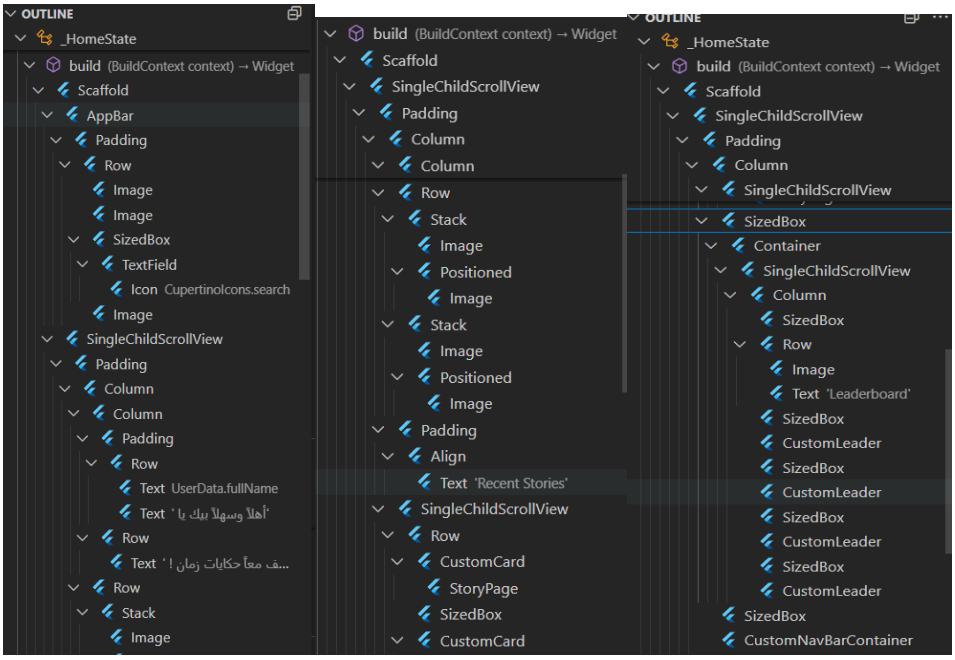
III. 'flutter/cupertino.dart'

IV. 'dart:convert'

- **Components (widgets) Explanation:**

1. **Scaffold:** Provides the structural layout, including the AppBar and Body.
2. **AppBar:** Customized to include icons for notifications, branding, and a search bar.
3. **FutureBuilder:** Handles asynchronous API requests and updates UI accordingly.
4. **SingleChildScrollView:** Allows vertical scrolling to accommodate various UI elements.
5. **Row & Column:** Organizes content in a structured manner for better user experience.
6. **CustomCard Widget:** Displays individual story cards with a topic, image, and navigation option.
7. **CustomLeader Widget:** Showcases leaderboard rankings, including profile images and points.
8. **Stack:** Used to layer images for aesthetic enhancement of the UI.
9. **Padding & SizedBox:** Provides proper spacing and alignment for UI components.
10. **CustomNavBarContainer:** Houses the navigation bar for seamless movement between pages.

-Widget Tree:



➢ **Stories Screen:**

A screen that displays various historical stories, allowing users to explore different eras through interactive cards. Users can navigate to a specific story page upon selection.

- **The main functionality**:

I. **API Request:**

Fetch stories from the backend API (https://gpfirst.vercel.app/).

Parse and display the received data dynamically.

Handle API errors gracefully and provide user feedback.

II. **Handling Loading State:**

Display a loading indicator while waiting for data.

Update the UI dynamically when data is successfully loaded.

Handle empty responses or errors properly.

III. **Navigation:**

Navigate to StoryPage upon selecting a story.

Allow navigation back using the AppBar button.

IV. **Custom Widgets:**

CustomStory Widget: A reusable component for displaying stories dynamically.

CustomNavBarContainer: A navigation bar to switch between screens conveniently.

V. **UI Customization & Design:**

Displays a **welcome message** using UserData.fullName.

Contains a **search bar** for future filtering.

Uses Row and Column to organize elements visually.

- **Libraries and Packages:**

V. 'flutter/material.dart'

VI. 'http'

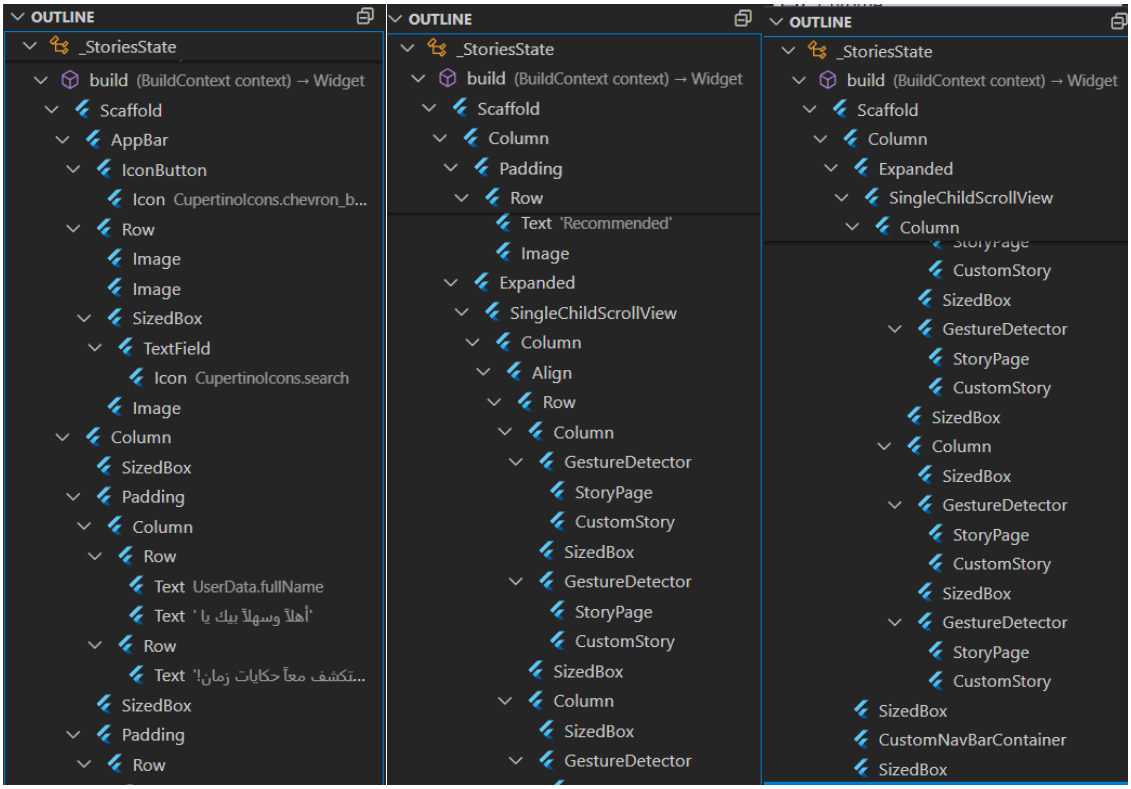
VII. 'flutter/cupertino.dart'

VIII. 'dart:convert'

- **Components (widgets) Explanation:**

1. **Scaffold:**Provides the basic structure of the page,Contains an AppBar and Body.
2. **AppBar:**Custom navigation button,A TextField for searching stories,Notification icon and logo images.
3. **CustomStory Widget:** Displays story details (title, description, rating, and images).,Handles user interactions via GestureDetector.
4. **GestureDetector:**Used to navigate to the StoryPage when a story is tapped.
5. **Row & Column:**Used for flexible layout structuring,Arranges story widgets neatly on the screen.
6. **CustomNavBarContainer:**Provides bottom navigation for seamless user experience.

-Widget Tree:



➢ **Upload File Screen:**

The Upload File screen allows users to upload a PDF document to the backend API. It validates the file type, processes the upload request, and provides feedback on the success or failure of the operation.

- **The main functionality**:

I. File Selection:

Users can select a PDF file using FilePicker.

Ensures only PDF files can be selected.

II. File Validation:

Verifies that the selected file is of type application/pdf.

Displays an error message if the file is invalid.

III. API Request for File Upload:

Sends a MultipartRequest to the backend API to upload the selected PDF file.

The API endpoint is https://gp-back-end-beige.vercel.app/api/data/uploadData.

IV. **Loading State Handling:**

Displays a CircularProgressIndicator while the upload is in progress.

Updates UI dynamically based on the upload status.

V. **User Feedback:**

Shows a success message when the file is uploaded successfully.

Displays an error message if the upload fails.

Notifies the user if no file is selected.

VI. **Navigation:**

Allows users to go back to the previous screen using the back button in the AppBar.

- **Libraries and Packages:**

I. 'flutter/material.dart'

II. 'http'

III. 'flutter/cupertino.dart'

IV. 'dart:convert'

V. 'Mime/mime.dart'

VI. 'file_piker'

Components (Widgets) Explanation:

1. **Scaffold:** Provides the basic structure of the screen with an AppBar and a centered body.
2. **AppBar:** Contains a back button for navigation Displays the screen title "Upload your PDF".
3. **Container (File Upload Box):**

Provides a UI for users to click and select a file.

Displays an upload icon and instructional text.

4. **CircularProgressIndicator:**

Shown while the file upload is in progress.

5. **Snackbar Notifications:**

Displays messages for file selection, upload success, and errors.

6. **Stack & Positioned:**

Used for decorative UI elements (logo and star image positioning)

Workflow:

1. User taps on the upload button.
2. A file picker dialog appears, allowing selection of a PDF.
3. If a file is selected:
    - MIME type is checked.
    - If valid, the file is uploaded via an HTTP request.
    - Displays success or failure messages.
4. If no file is selected or an invalid file is chosen:
    - An error message is displayed.
5. After upload, feedback is shown and UI is updated.

-Widget Tree:

